SP1 zkVM Architecture

Succinct Labs

April 29, 2026

Overview

This document describes the architecture of the **SP1 zkVM** according to the W1 deliverable of the EF zkVM whitepaper guidelines[1]. SP1 proves execution of RISC-V guest programs using:

- Execution of RV64IM programs encoded as ELF binaries
- Sharded execution traces with segmentation logic by trace area estimation.
- Specialized computational units at the arithmetization layer called "chips", whose constraints enforce correct execution of RISC-V instructions and cryptographic operations.
- Local bus interactions (LogUp + GKR) for inter-chip communication within each shard.
- Global bus interactions across shards (with septic curve accumulation).
- A multilinear proof system ("Hypercube") with BaseFold polynomial commitments over the KoalaBear field.
- Recursive aggregation via normalize / compose / shrink circuits

1 Execution Model

We define how a program is executed.

1.1 Program, Execution, and VM State

A **program** $\mathcal{P}$ is a RISC-V ELF binary targeting the RV64IM instruction set (64-bit base integer with multiply/divide extension).

The program $\mathcal{P}$ operates on the **VM state** S, defined as:

- `pc`: 64-bit program counter
- `regs`[0..31]: 32 general-purpose registers (x0–x31), each 64 bits wide, with x0 hardwired to zero
- `mem`[]: byte-addressed writable memory

[1] `https://zkevm.ethereum.foundation/blog/cryptography-research-update`

We separately denote the code of the program by `code`. It is immutable and read-only, and is not part of the VM state.

A single instruction step of the program execution is defined as a transition from one VM state to another according to the semantics of the instruction op retrieved from `code` at the program counter $\mathtt{pc}_1$. This is denoted as follows:

$$S := (\mathtt{pc}_1, \mathtt{regs}_1, \mathtt{mem}_1) \xrightarrow{\text{op}:=\mathtt{code}[\mathtt{pc}_1]} S' := (\mathtt{pc}_2, \mathtt{regs}_2, \mathtt{mem}_2) \tag{1}$$

For example, if op is an addition instruction, which adds the first two registers and stores in the third register, then the semantics of the step is defined as

$$\mathtt{pc}_2 = \mathtt{pc}_1 + 4, \tag{2}$$

$$\mathtt{regs}_2 = \mathtt{regs}_1 \text{ except that } \mathtt{regs}_2[2] := \mathtt{regs}_1[0] + \mathtt{regs}_1[1], \tag{3}$$

$$\mathtt{mem}_2 = \mathtt{mem}_1. \tag{4}$$

We will use the notation $(\mathtt{pc}_1, \mathtt{regs}_1, \mathtt{mem}_1) \to (\mathtt{pc}_2, \mathtt{regs}_2, \mathtt{mem}_2)$ to denote that S transitions to S' by executing instruction op $:= \mathtt{code}[\mathtt{pc}_1]$.

An **execution trace** for program `code` applied to inputs $\mathcal{I}$ is a sequence of VM states

$$\mathcal{T}: \quad S_0 \to S_1 \to \cdots \to S_T$$

where each step corresponds to execution of one instruction.

A **zkVM proof** for $\mathcal{P}$ and initial state S_0 is a proof of the existence of an execution trace $\mathcal{T}$ as above with S_0 as the initial state.

Supported Instructions. SP1 supports the full RV64IM instruction set[2]: the base 64-bit integer instructions (I) and the integer multiply/divide extension (M).

Execution Environment. Additionally, SP1 provides *precompiles*—accelerated implementations of expensive operations—invoked via the RISC-V `ecall` instruction with a syscall code in register `t0`.

The precompiles include standard system-level ecalls—halt, I/O (write, hint read/len, commit), commit— as well as the following accelerated cryptographic operations (we include here only the precompiles relevant to the ethereum proving usecase):

- **Hash functions:** SHA-256 (extend + compress), Keccak-256 permutation, Poseidon2.
- **Elliptic curve operations:** point addition, doubling, and decompression for Secp256k1, Secp256r1, and BN-254.

Predicates. For our purpose the exact semantics of each instruction is not important; we treat them as black boxes with predicates modeling correctness. For a given instruction op, the predicate φ_{op} has the semantics

$$\varphi_{op}(\mathtt{pc}_1, \mathtt{regs}_1, \mathtt{mem}_1, \mathtt{pc}_2, \mathtt{regs}_2, \mathtt{mem}_2) = \text{TRUE} \iff (\mathtt{pc}_1, \mathtt{regs}_1, \mathtt{mem}_1) \xrightarrow{\text{op}} (\mathtt{pc}_2, \mathtt{regs}_2, \mathtt{mem}_2) \wedge \mathtt{code}[\mathtt{pc}_1] = op. \tag{5}$$

[2] `https://riscv.org/technical/specifications/`

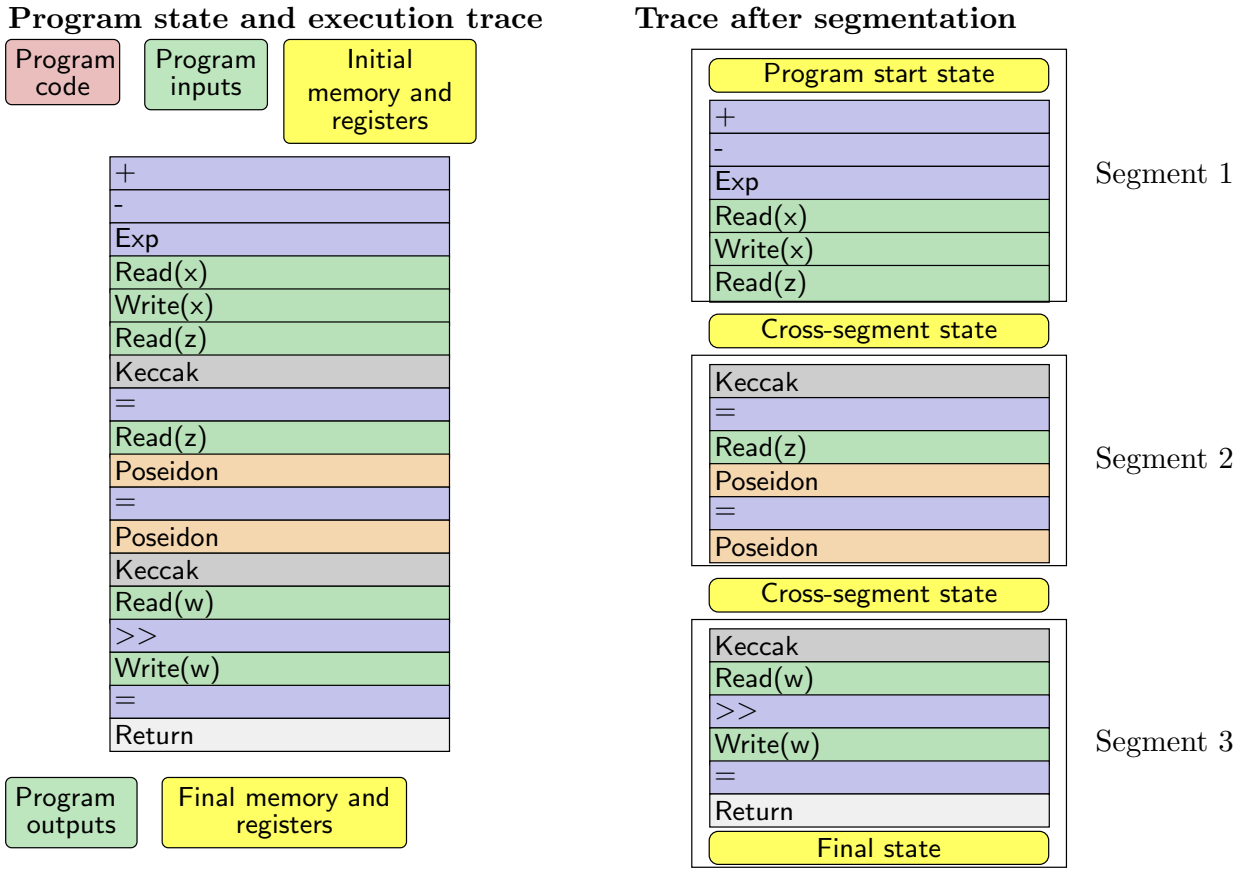

Figure 1: Segmentation of the execution trace into 3 segments

2 Segmentation

To facilitate efficient proving of large computations, SP1 splits the execution into "shards". We now explain how SP1 undertakes this splitting.

2.1 Execution Shards

After execution, the execution trace $\mathcal{T}$ is divided into *shards* $\mathcal{G}_1, \mathcal{G}_2, \ldots, \mathcal{G}_m$ (cf. Fig. 1). Shards are not fixed-size; instead, the executor splits the trace when the estimated trace area of the current shard exceeds a configurable threshold or more rarely if any one opcode is invoked more than a fixed number of times[3]. Each shard $\mathcal{G}_i$ corresponds to a contiguous subsequence of the execution trace, from S_{start_i} to S_{end_i}, and is proven by its own core shard proof.

Sharding reduces the proof of the entire execution to proving the validity of each shard and consistency between them. The global consistency argument (enforced during recursive aggregation) asserts:

$$S_{\text{end}_i} = S_{\text{start}_{i+1}} \text{ for all } i \in [m-1], \quad S_{\text{start}_1} = S_0, \quad S_{\text{end}_m} = S_T.$$

[3]The default element threshold is $2^{28} + 2^{27}$, and the maximum opcode count is 2^{22}.

2.2 Shard layout

Each shard is proven by a *chip cluster*—a collection of chips (specialized constraint systems) that are included together in a single proof. The cluster determines which chips are present and thus the circuit shape. Within a shard, each chip produces a table of field elements with algebraic constraints on its columns. Ron: Suggest: Within a shard, each chip is represented by a table. The columns corresponds to the operands relevant to the given chip, as well as auxiliarly "helper columns". The rows correspond to the different executions of the instruction in the given shard. The shard is further associated with algebraic constraints (AIRs) between its columns, which are designed to check correctness (via the zerocheck protocol, described below).

Consistency between chips is enforced by a *bus*: a LogUp interaction argument (proved via GKR) Ron: can we add a bibliography? happy to take care of it that checks that all "send" and "receive" messages across chips balance within the shard. Ron: worth mentioning that the bus is also used to enfore local constraints on the chip as well e.g. range checks All inter-shard communication passes through the global accumulation bus, arithmetized as a `Global` table present in all shards.

Each shard proof attests to the relation

$$\mathcal{R}^{(C)}_{\text{shard}} = \left\{ ((\mathsf{pv}, \sigma);\ \mathsf{trace}) \;\middle|\; \begin{array}{c} \text{All chip constraints in cluster } C \text{ are satisfied} \\ \text{All local-scope bus interactions balance (via LogUp)} \\ \sigma \text{ is the correct global cumulative sum} \\ \text{Public values } \mathsf{pv} \text{ are consistent with the trace} \end{array} \right\}$$

where C is the chip cluster determining the circuit, pv are the shard's public values (boundary state, digests, etc.), and σ is the shard's global cumulative sum[4].

The clusters are of four types:

1. **Base core cluster** (`core`). Contains all RISC-V instruction chips (ALU, memory load/-store, control flow, etc.) together with infrastructure chips: `Program` (instruction lookup), `ByteLookup` and `RangeLookup` (preprocessed lookup tables), `MemoryLocal` (local memory state), `MemoryBump` and `StateBump` (timestamp management), `SyscallCore`, `SyscallInstrs` (syscall dispatch), and `Global` (global bus accumulator).

2. **Extended core clusters** (`core+`P). Some precompiles are compact enough to be retained in the core cluster, avoiding the overhead of a separate shard proof and allowing communication with their calling context via the local lookup argument. The retainable extensions are: `MemoryGlobalInit`/`MemoryGlobalFinal`, `Bls12381Fp`, `Bn254Fp`, `Sha256`{`Extend`,`Compress`} (with their control chips), `Uint256Ops`, and `Poseidon2`. Extended clusters are generated for subsets of size 0, 1, or all of these extensions, plus one additional combination (`core` + memory global + SHA-256 + `Uint256Ops`).

3. **Deferred precompile clusters** (`precompile-`P). Precompiles with high arithmetization costs (in terms of number of columns in the arithmetization) are proved in their own shards. Each such cluster contains the precompile-specific chip(s) together with a base set of infrastructure chips: `Program`, `ByteLookup`, `RangeLookup`, `SyscallPrecompile`, `MemoryLocal`, and `Global`. They communicate with their RISC-V calling context via the global lookup argument. The deferred precompiles include: elliptic curve operations (Ed25519, Secp256k1, Secp256r1, BN-254, BLS12-381 point add/double), field arithmetic (BLS12-381 and BN-254

[4]The proof system for this circuit is named "core" in soundcalc

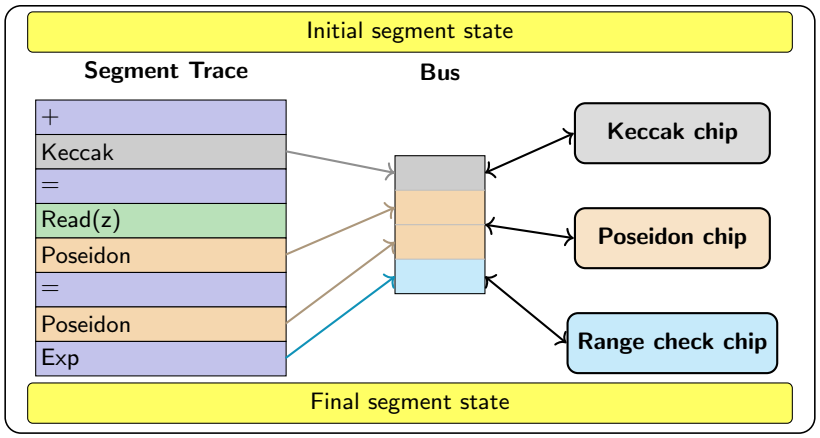

Figure 2: Segment structure: segment trace connects to the bus which then connects to chips.

$\mathbb{F}_p/\mathbb{F}_{p^2}$ operations), hash functions (Keccak permutation, SHA-256, Poseidon2), and big-integer arithmetic (`Uint256Mul`, `Uint256Ops`).

4. **Memory boundary cluster** (`mem-boundary`). The global memory argument requires initialization and finalization: sending the initial memory state into the global bus and receiving the final memory state from the global bus. This cluster contains `MemoryGlobalInit`, `MemoryGlobalFinal`, `Global`, and the preprocessed chips. Note that when memory initialization/finalization is retained into an extended core cluster (type 2), this standalone cluster is not needed (this happens for very small programs so that the proof can fit into one shard).

3 Correct execution of a program

In this section, we define via predicates what it means that a program is correctly executed. We will use these predicates in Section 4 to define the relations that the zkVM proves.

3.1 Predicates for individual operations

Each operation is treated as a black box with its own predicate φ_{op} (cf. (5)). Throughout this section, subscripts 1 and 2 denote states before and after an operation. We write $S_1 := (\texttt{pc}_1, \texttt{regs}_1, \texttt{mem}_1)$ and $S_2 := (\texttt{pc}_2, \texttt{regs}_2, \texttt{mem}_2)$ for the states before and after the operation.

We group operations as follows:

- **ALU operations.** Integer arithmetic (ADD, SUB, MUL, DIV, REM, and their variants), bitwise operations (AND, OR, XOR), shifts (SLL, SRL, SRA), and comparisons (SLT, SLTU). Each has a predicate $\varphi_{\text{op}}(S_1, S_2)$.
- **Memory operations.** Loads (LB, LH, LW, LD and unsigned variants) and stores (SB, SH, SW, SD). Load: $\varphi_{\text{load}}(S_1, S_2)$ — reads memory into a register. Store: $\varphi_{\text{store}}(S_1, S_2)$ — writes a register value to memory.
- **Control flow.** Branches (BEQ, BNE, BLT, BGE, BLTU, BGEU), jumps (JAL, JALR), and upper-immediate (LUI, AUIPC).
- **Precompiles.** Invoked via `ecall`; each precompile P has a predicate $\varphi_{\text{P}}(S_1, S_2)$ covering all state changes (register and memory) produced by the syscall.

3.2 Correct execution of a single step

A single step executes the instruction at the current program counter. The step predicate selects the matching operation via a disjunction over all opcodes:

$$\varphi_{\text{step}}(S_1, S_2) := \bigvee_{\text{op} \in \text{operations}} \varphi_{\text{op}}(S_1, S_2)$$

where S_1 and S_2 are the states before and after the step.

3.3 Chip interactions and the bus

Within a shard, chips communicate through a *bus*: a collection of typed messages that chips *send* (produce) and *receive* (consume). For example, when the execution trace chip encounters a precompile call, it sends the call's inputs to the bus; the corresponding precompile chip receives those inputs, performs the computation, and sends the results back. Consistency is enforced by requiring that, for each interaction kind, the multiset of sent messages equals the multiset of received messages.

SP1 implements this via a **LogUp argument** (proved via GKR): each chip contributes terms $+1/(\alpha - m_i)$ for sends and $-1/(\alpha - m_j)$ for receives, where α is a random challenge and m_i, m_j are message fingerprints. The sum must equal zero, which holds if and only if sends and receives match as multisets Ron: (whp over α).

Each interaction is declared with one of two *scopes*:

- **Local** interactions must balance within the shard. This is enforced by the LogUp + GKR Ron: minor but I think either LogUpGKR or LogUp/GKR look nicer argument inside the shard proof.
- **Global** interactions need not balance within a single shard. Instead, each shard computes its net contribution $\sigma \in E(\mathbb{F})$, a point on a degree-7 elliptic curve over the base field (the "septic curve"), and outputs it as a public value. The contributions are accumulated across all shards during recursion, and the final completeness check asserts $\sum_i \sigma_i = \mathcal{O}$ (the identity point).

The global scope is used for interactions that cross shard boundaries, most notably memory consistency and precompile delegation to separate shards.

3.4 Memory consistency

SP1 does *not* use Merkle tree commitments for memory. Instead, memory consistency is enforced entirely through the global bus. Ron: via the memory-checking argument of Blum et al.

At the beginning of execution, a dedicated `MemoryGlobalInit` chip sends each initial memory cell $(addr, value, timestamp)$ into the global bus. At the end of execution, a `MemoryGlobalFinal` chip receives the final memory state from the global bus. During execution, every load and store instruction interacts with the bus: a load receives the current value at the given address and sends an updated timestamp; a store receives the old value and sends the new value with an updated timestamp.

Because these are global-scope interactions, they accumulate across shards via the septic curve. The final completeness check ($\sigma = \mathcal{O}$) ensures that every memory cell written is eventually read with the correct value, and that the initial and final memory states are consistent with the full execution.

Within a shard, local memory interactions (via the `MemoryLocal` chip) handle the read/write operations that occur entirely within that shard, using the local LogUp argument for consistency.

4 Recursion Architecture

In the previous two sections, we described how a proof of correct RISC-V execution is produced by dividing the computation into shards, arithmetizing shard correctness via constraints and lookups, and finally gluing together the shard proofs via a global bus consistency argument. The number of shards scales roughly with the cycle count of the execution; to produce constant-size proofs it is therefore necessary to introduce some form of recursion or proof aggregation. In this section, we describe the approach taken by SP1 to address this problem.

It is useful when discussing recursion to set some terminology to avoid the confusion caused by "the recursive nature of recursion" Ron: ?. First, by a **recursion program** or **recursion circuit**, we mean a program or arithmetic cirucit that verifies one or more SNARK proofs and also optionally checks some aggregation logic. SNARK proofs can typically be verified in the circuit model of computation, so the term "circuit" here is suitable. Given a recursion circuit $\mathcal{C}$, when we say a $\mathcal{C}$ proof, we mean a proof of the correct execution of $\mathcal{C}$. Often, the inputs to a recursion circuit are $\mathcal{C}'$ proofs for some other circuit $\mathcal{C}'$; we advise the reader to be aware that the term $\mathcal{C}$ proof refers to the *output* of proving $\mathcal{C}$. Finally, given a collection

4.1 Topology

The recursion proof pipeline consists of multiple circuit types, summarized in the table below. [5]

Circuit	Purpose	Relation
`normalize`	1-to-1 shape normalization	$\mathcal{R}_{\text{norm}}$
`compose`	N-to-1 aggregation ($1 \leq N \leq 4$)	$\mathcal{R}_{\text{compose}}$
`shrink`	Final proof size reduction	$\mathcal{R}_{\text{shrink}}$

In more detail:

- `normalize`: A 1-to-1 circuit that verifies a single core shard proof. A `normalize` proof is always of a fixed shape (which we call `compress_shape`), irrespective of the input proof's shape (Sec. 4.2).
- `compose`: An N-to-1 recursion circuit ($2 \leq N \leq 4$) Ron: maybe $2 \leq N \leq 4$? that verifies multiple proofs of shape `compress shape` and performs some aggregation logic. A `compose` proof is always a single proof of shape `compress_shape` (Sec. 4.3).
- `shrink`: A 1-to-1 circuit that takes the final complete `compose` proof and re-proves it with smaller FRI parameters, reducing the proof size in preparation for the outer proof system (Sec. 4.4).

We visualize the recursion topology in Fig. 3. At the leaves, `core` shard proofs are each normalized to `compress_shape` by a `normalize` circuit. The resulting proofs are then aggregated by a tree of `compose` circuits (fan-in up to 4). We refer to the `normalize` and `compose` layers collectively as the "compress" stage. The compress stage terminates when all proofs have been aggregated into a single complete `compose` proof. Finally, this proof is passed to the `shrink` circuit, which re-proves it with reduced FRI parameters to minimize proof size before conversion to the outer proof system.

[5] The proof parameters for `normalize` and `compose` are the same, so in soundcalc, they are listed under the name "compress". The proof parameters for `shrink` are listed in soundcalc under "shrink".

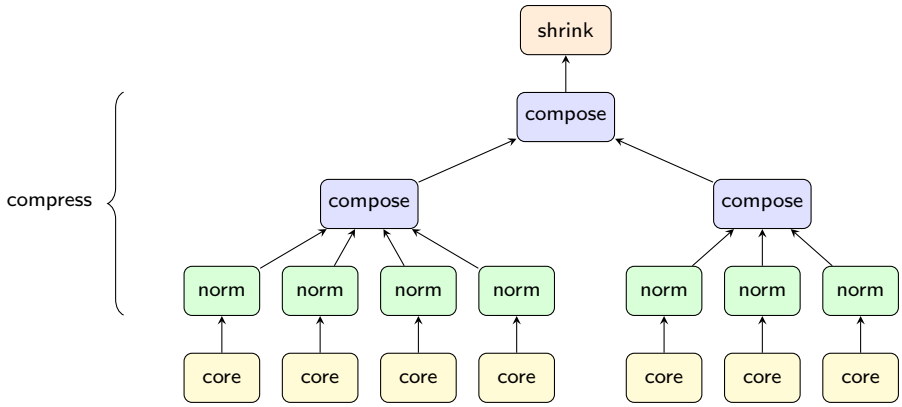

Figure 3: Recursion topology. Colors: core , normalize , compose , shrink . The tree is illustrative; the actual fan-in and depth depend on the number of shards.

Programs vs. verifiers. Because of recursive proof aggregation, each recursion step acts as both a *prover* and a *verifier*: the program executed by the prover contains the verifier for the previous step's proof system. For example, the `compose` program contains the compress verifier, which verifies `normalize` and `compose` proofs. In the subsections below, when we say "the `compose` circuit verifies a proof," we mean that the compose *program* embeds the compress *verifier* as part of its constraint system.

Verifying key Merkle tree. Each recursion circuit type may have multiple *shapes*—distinct circuit configurations that arise from different chip clusters and trace dimensions. A verifying key vk is derived for each shape, yielding approximately 181,000 distinct keys in total. These keys are organized into a Merkle tree whose root vk_root is used to authenticate recursion proofs.

The shapes are enumerated as follows:

- **Normalize** (the vast majority of leaves): one shape per combination of chip cluster, preprocessed and main stacked BaseFold column counts, and padding column counts.
- **Compose**: one shape per arity ($N = 2, 3, 4$).
- **Shrink**: a single shape.

The `compose` program enforces VK authentication: it takes Merkle membership proofs for each input proof's verifying key as part of its witness, verifies them against the Merkle root, and checks that every input proof's rpv.vk_root equals this root. The `normalize` program does not perform VK membership checks itself; it propagates vk_root as a public value, which the compose verifier then checks for consistency.

Note that, because of the 1-1 normalization, there truly is exactly one `compose` circuit for each arity. If not for the normalization, the number of arity N `compose` circuits would be C^N, where C is the number of core shard proof shapes.

4.2 `normalize` circuit

The `normalize` circuit proves the correct verification of a single core shard proof. Its output is a proof of a fixed shape (`compress_shape`), irrespective of the input shard proof's shape (which varies with the chip cluster). The circuit takes as witness a shard proof π, the verifying key vk, a completeness flag is_complete $\in \{0, 1\}$, and a VK Merkle root vk_root.

The circuit performs two categories of work:

- **Shard proof verification.** A Fiat–Shamir challenger is initialized by observing the verifying key, and the core shard proof is verified against this challenger (polynomial commitment openings and constraint satisfaction).
- **Aggregation checks.** Outside the shard verifier, the circuit enforces cross-shard and global correctness:
  - *First-shard PC validation.* If the shard is the first execution shard, the circuit asserts that $\mathsf{pc_start}$ matches the verifying key's start PC.
  - *Global cumulative sum accumulation.* The output global cumulative sum σ is computed by adding the verifying key's $\mathsf{initial_global_cumulative_sum}$ (if this is the first shard, otherwise $\mathcal{O}$) to the shard's own $\mathsf{global_cumulative_sum}$.
  - *Public values propagation.* The shard's public values (boundary PCs, timestamps, memory addresses, digests, exit codes, etc.) are propagated into a `RecursionPublicValues` struct together with the VK digest and VK Merkle root. A Poseidon-2 digest of these values is committed as the proof's public output.
  - *Completeness conditions.* When $\mathsf{is_complete} = 1$, the proof is expected to attest to a full program execution rather than a single shard, so final boundary conditions are checked — including that the program has halted, the first shard was included, all "previous" boundary values are at their initial state, and the global cumulative sum equals $\mathcal{O}$ (the identity point on the septic curve).

Formally, let $\Pi^{(C)}_{\text{shard}}$ be the SNARK for $\mathcal{R}^{(C)}_{\text{shard}}$. Write pv for the shard's public values (including its global cumulative sum σ_{shard}) extracted from the proof, and rpv for the recursion public values output by the circuit.

$$\mathcal{R}_{\text{norm}} = \left\{ \mathsf{rpv};\ (\pi,\ \mathsf{vk},\ C) \;\middle|\; \begin{array}{c} \Pi^{(C)}_{\text{shard}}.\mathsf{Verify}(\mathsf{pv},\ \pi) = 1 \\ \mathsf{rpv.contains_first_shard} = 1 \implies \mathsf{pv.pc_start} = \mathsf{vk.pc_start} \\ \mathsf{rpv}.\sigma \;=\; \sigma_{\text{init}} + \mathsf{pv}.\sigma_{\text{shard}}, \quad \sigma_{\text{init}} := \begin{cases} \mathsf{vk}.\sigma_0 & \text{if first shard} \\ \mathcal{O} & \text{otherwise} \end{cases} \\ \mathsf{rpv} \text{ is correctly derived from } \mathsf{pv},\ \mathsf{vk},\ \text{and auxiliary inputs} \\ \mathsf{rpv.is_complete} = 1 \implies \varphi_{\text{complete}}(\mathsf{rpv}) \end{array} \right\}$$

where $\varphi_{\text{complete}}$ asserts the final boundary conditions described above (program halted, first shard included, initial boundary values zero, $\sigma = \mathcal{O}$).

4.3 `compose` circuit

The `compose` circuit is an N-to-1 recursion circuit ($1 \le N \le 4$) that consumes N proofs — each of which may be a `normalize` or `compose` proof — and produces one proof of the fixed shape `compress_shape`. Its purpose is to aggregate a batch of proofs that cover contiguous portions of the execution into a single proof.

The circuit performs two categories of work:

- **Proof verification.** Each input proof π_i is verified against its own verifying key vk_i. The circuit checks that every proof's $\mathsf{vk_root}$ matches a single witnessed value, and that each proof's recursion public values have a valid digest.

- **Continuity and aggregation checks.** The circuit enforces that the N proofs form a contiguous chain:

  - *Sequential continuity.* For each consecutive pair of proofs, the "last" boundary values of proof i (PC, timestamp, memory addresses, page indices, exit code, syscall flags, etc.) must equal the "previous" boundary values of proof $i{+}1$.
  - *Program identity.* The $\mathsf{sp1_vk_digest}$ and $\mathsf{proof_nonce}$ are asserted to be identical across all N proofs.
  - *Accumulation.* The global cumulative sums σ_i from all proofs are summed (via elliptic curve addition on the septic curve). The shard counts and $\mathsf{contains_first_shard}$ flags are accumulated.
  - *Completeness conditions.* When $\mathsf{is_complete} = 1$, the same final boundary conditions as in the `normalize` circuit are checked via $\varphi_{\text{complete}}$.

Formally, let Π_{comp} denote the SNARK for `compress_shape` proofs (used by `normalize` and `compose` alike). Write rpv_i for the recursion public values of the i-th input proof ($1 \le i \le N$, $1 \le N \le 4$), and rpv for the aggregated output.

$$\mathcal{R}_{\text{compose}} = \left\{ \mathsf{rpv};\ (\{\mathsf{vk}_i, \pi_i\}_{i=1}^{N},\ \mathsf{vk_root}) \;\middle|\; \begin{array}{c} \forall\, i:\ \Pi_{\text{comp}}.\mathsf{Verify}(\mathsf{rpv}_i,\, \pi_i) = 1 \ \wedge\ \mathsf{rpv}_i.\mathsf{vk_root} = \mathsf{vk_root} \\ \forall\, i < N:\ \mathsf{rpv}_i.\mathsf{last_*} = \mathsf{rpv}_{i+1}.\mathsf{prev_*} \\ \forall\, i:\ \mathsf{rpv}_i.\mathsf{sp1_vk_digest} = \mathsf{rpv}_1.\mathsf{sp1_vk_digest} \\ \mathsf{rpv}.\sigma \ = \ \sum_{i=1}^{N} \mathsf{rpv}_i.\sigma \\ \mathsf{rpv} \text{ boundary values derived from } \mathsf{rpv}_1.\mathsf{prev_*} \text{ and } \mathsf{rpv}_N.\mathsf{last_*} \\ \mathsf{rpv}.\mathsf{is_complete} = 1 \implies \varphi_{\text{complete}}(\mathsf{rpv}) \end{array} \right\}$$

Here "$\mathsf{last_*} = \mathsf{prev_*}$" is shorthand for the continuity of all chained boundary values (PC, timestamp, memory init/finalize addresses, page indices, exit code, and syscall flags).

4.4 `shrink` circuit

The `shrink` circuit is a 1-to-1 recursion circuit that takes the final complete `compose` proof and re-proves it with reduced FRI parameters. Its purpose is to reduce the proof size: the compress stage uses FRI parameters sized for a log-trace-area of 27, while the shrink circuit uses parameters for a log-trace-area of 25, yielding a significantly smaller proof that is cheaper to verify in the subsequent outer proof system (e.g., Groth16). The shrink circuit has a single fixed shape.

The circuit takes as witness a compress-stage proof π, the verifying key vk of the input proof, and a VK Merkle membership proof mkl.

The circuit performs the following work:

- **VK Merkle verification.** The input proof's verifying key is checked against the VK Merkle tree, ensuring it belongs to the set of permitted recursion circuits.

- **Proof verification.** The compress-stage proof π is verified against its verifying key vk using the compress verifier (the same verifier used inside `compose`).

- **Completeness enforcement.** The circuit asserts that the input proof is marked $\mathsf{is_complete} = 1$, meaning it attests to a full program execution—the program has halted, all shards have been aggregated, the global cumulative sum equals $\mathcal{O}$, and all boundary conditions are satisfied.

- **Output construction.** The output recursion public values are derived from the input proof's public values. A Poseidon-2 digest of the output public values is committed as the shrink proof's public output.

Formally, let Π_{comp} denote the SNARK for `compress_shape` proofs (as in the `compose` circuit). Write rpv' for the recursion public values extracted from the input proof and rpv for the output.

$$\mathcal{R}_{\text{shrink}} = \left\{ \mathsf{rpv};\ (\mathsf{vk},\ \pi,\ \mathsf{mkl}) \ \middle|\ \begin{array}{l} \Pi_{\text{comp}}.\mathsf{Verify}(\mathsf{rpv}',\ \pi) = 1 \\ \mathsf{vk} \text{ verified against } \mathsf{mkl} \\ \mathsf{rpv}'.\mathsf{is_complete} = 1 \\ \varphi_{\text{complete}}(\mathsf{rpv}') \\ \mathsf{rpv} \text{ is correctly derived from } \mathsf{rpv}' \end{array} \right\}$$

Because shrink enforces completeness, its output serves as the definitive attestation that the entire program execution is valid. The shrink proof is then passed to the outer proof system for final verification.

5 Proof system instantiations

In the previous sections we described how to arithmetize correct RISC-V execution by decomposing it into a collection of tables of field elements, each with algebraic constraints on its columns, and allowing tables to *send* and *receive* messages with multiplicity through a collection of interaction kinds. All tables in a chip cluster are zero-padded to a common height $\mathcal{N}_{row} = 2^{n_{row}}$; for core circuits $n_{row} = 22$. Throughout this section, let W denote the total number of columns across all tables in the shard.

The proof of each shard follows a four-step protocol:

1. **Commit.** Treat the unpadded trace data as a single dense multilinear polynomial Q and commit to it via Ron: (interleaved) Reed–Solomon encoding and Merkle hashing (Sec. 5.2).

2. **LogUp + GKR.** Prove that all local-scope interactions balance (sends = receives as multisets). The output is a collection of W evaluation claims on individual trace columns, all at a single point η (Sec. 5.3).

3. **Zerocheck + Sumcheck.** Ron: i sugggest to remove sumcheck from the title Prove that every algebraic constraint is satisfied on all unpadded rows. Batch the constraint zerocheck with the LogUp+GKR opening claims into a single sumcheck whose output is a new collection of W evaluation claims at a point η' (Sec. 5.4).

4. **PCS opening.** Reduce the W column claims at η' to a single claim on Q via the jagged PCS (Sec. 5.5), then prove the claim on Q via BaseFold (Sec. 5.6).

Because every table is padded to the same height $\mathcal{N}_{row}$, the verifier can be described as an arithmetic circuit that depends on the chip cluster and the total unpadded trace area, but is *independent* of the individual tables' unpadded heights. The total number of distinct verifier

circuits is therefore bounded by $\approx 2^{18}$, which is small enough that all circuits can be pre-built; no VM is needed in the recursion layer.

If the prover actually materialized the full $W \times \mathcal{N}_{row}$ padded trace, the data volume would be enormous: $W \approx 2^{12}$ columns at 4 bytes each gives $4 \times 2^{12} \times 2^{22} = 2^{36}$ bytes $= 64\,\text{GB}$. The jagged PCS (Sec. 5.5) ensures that both prover and verifier costs depend only on the size of the *unpadded* data.

Notation. Whenever a variable appears in `typewriter font`, it refers to the corresponding parameter in the `sp1.toml` configuration file in the `soundcalc` repository.

5.1 Field and Fiat–Shamir challenger

Base field. SP1 operates over the **KoalaBear** prime field $\mathbb{F}_p$ with $p = 2^{31} - 2^{24} + 1$.

Extension field. All verifier randomness is sampled over the degree-4 extension

$$\mathbb{F}_{p^4} \;=\; \mathbb{F}_p[x]/(x^4 - 3),$$

a binomial extension. We write $\mathbb{F}$ for the base field and $\mathbb{E}$ for the extension when the distinction matters (`field = "KoalaBear^4"` in `sp1.toml`).

Fiat–Shamir challenger. Interactive challenges are derived non-interactively via a **Poseidon-2 duplex sponge**. The sponge state has width 16 base-field elements; a digest consists of 8 base-field elements ($8 \times 31 = 248$ bits; cf. `hash_size_bits = 248`).

Proof-of-work grinding. At three points in the protocol the prover must perform proof-of-work grinding before the verifier supplies the next challenge:

- Before batching BaseFold evaluation claims `grinding_batching_phase` bits.
- Before the FRI query phase in BaseFold: `grinding_query_phase` bits.
- Before sampling the randomness used for LogUp fingerprinting (`grinding_bits_lookup`).

These grinding steps raise the cost of a forging attempt without increasing verifier complexity.

5.2 Trace commitment

All unpadded table data is flattened into a single array. This array is divided into chunks of size `dense_length` (a power of two); each chunk becomes one "column" of a dense multilinear polynomial Q. There are `dense_batch` such chunks in total, so Q is a multilinear polynomial in $\log_2(\texttt{dense_length}) + \lceil \log_2(\texttt{dense_batch}) \rceil$ variables.

The prover commits to Q as follows:

1. Interpret each chunk as the evaluations of a univariate polynomial Ron: not as coefficients? IIRC for basefold we want the coefficient representation + this saves the inverse FFT of degree $= \texttt{dense_length} - 1$.
2. Reed–Solomon encode each chunk at rate $\rho = 1/\texttt{blowup_factor}$, producing codewords of length $\texttt{dense_length} \times \texttt{blowup_factor}$.

3. Hash the encoded columns into a single Merkle tree; the Merkle root constitutes the trace commitment. Ron: maybe say that the rows are the leaves of the Merkle tree?

The prover also reports the unpadded height h_i of each table. Together with the commitment, these heights fully determine the trace.

5.3 LogUp + GKR

The goal of this step is to prove that all *local-scope* interactions balance: for each interaction kind, the multiset of sent messages equals the multiset of received messages. (Recall from Sec. 3.3 that global interactions are handled by the septic curve accumulator.)

LogUp equation. A chip cluster defines a maximum number `num_columns_S` of interactions found in the chips, and a maximum message size `num_lookups_M` for all chips Each message $\mathbf{v} = (v_1, \ldots, v_{\texttt{num_lookups_M}})$ sent through the bus with index BI is mapped to a field element via a *multilinear fingerprint*:

$$\mathrm{fp}(\mathbf{v}) \;=\; \mathrm{eq}_\beta(0) \cdot BI + \sum_{i=1}^{\texttt{num_lookups_M}-1} \mathrm{eq}_\beta(i)\, v_i \qquad (\texttt{multilinear_fingerprint = true}),$$

where $\beta \in \mathbb{E}^{\lceil \log_2()\texttt{num_lookups_M})\rceil}$ is randomness sampled for the LogUp protocol. For a given interaction, let t denote the vector $\{\mathrm{fp}(\mathbf{v}_x)\}_{x\in\{0,1\}^{n_{row}}}$, where $\mathbf{v}_x$ is the message passed from row $x \in \{0,1\}^{n_{row}}$. Similarly, let $m(x)$ denote the multiplicity of the message at row x. The verifier samples $\alpha \in \mathbb{E}$ and the prover computes and sends

$$\kappa := \sum_{x\in\{0,1\}^{n_{row}}} \frac{m(x)}{\alpha - t(x)}. \tag{6}$$

The verifier sums the κ values across the interactions in the chip cluster, and rejects a proof if the sum is non-zero (the consistency of bus sends and receives is equivalent to this sum being zero).

GKR reduction. The above fractional sums can be described as the outputs of a circuit of depth n_{row}; SP1 proves the correctness of this circuit evaluation via the GKR polynomial IOP Ron: interactive proof, not a PIOP.

Output. After n_{row} layers the GKR protocol terminates with evaluation claims on the multilinear extensions $\hat{m}(\eta)$ and $\hat{t}(\eta)$. The original vectors m and t are affine-linear combinations of the original columns of the chip. The prover provides column openings of the chip at η, and the verifier checks that the proper affine-linear combinations of the chips match the claimed values for $\hat{m}(\eta), \hat{t}(\eta)$.

5.4 Zerocheck

The goal of this step is to prove that every algebraic constraint is satisfied on the unpadded rows of every table, while simultaneously proving the column evaluation claims produced by LogUp+GKR.

Setup. The chip cluster has k tables. Table i has w_i columns $p_{i,1}, \ldots, p_{i,w_i}$, unpadded height h_i, and c_i constraints $C_{i,1}, \ldots, C_{i,c_i}$, each of total degree at most $\texttt{air_max_degree} = 3$.

Zerocheck sumcheck. The constraint and GKR opening claims are batched (via random challenges) into a single sumcheck over $\{0,1\}^{n_{row}}$. After n_{row} rounds, the sumcheck reduces to an evaluation claim on the batched polynomial at a random point η'. This claim involves the constraint polynomials $C_{i,j}$ evaluated at the columns of each table at η'. Since each $C_{i,j}$ is an explicit low-degree polynomial, the verifier can check the claim given the individual column values $p_{i,\ell}(\eta')$. The prover therefore supplies these values, yielding W column evaluation claims at η'.

5.5 Jagged PCS

The goal of this step is to reduce the W column evaluation claims at η' to a single claim on the dense polynomial Q committed in Sec. 5.2.

Column $\leftrightarrow$ dense relationship. Each padded column p_ℓ of height $\mathcal{N}_{row}$ is a multilinear polynomial in n_{row} variables. Its unpadded entries occupy a contiguous slice of the dense array underlying Q. The jagged PCS[6] bridges the two representations without the prover ever materializing the padded columns. The protocol can be described as a sumcheck over a product of multilinears, one of which is Q and the other of which can be efficiently evaluated. We use the jagged assist (as described in the reference Ron: missing) to further reduce verifier work in the evaluation of the latter polynomial.

Output. The jagged PCS reduces the W column claims to a single claim $Q(z) = v$ on the dense polynomial.

5.6 Dense PCS (BaseFold)

The final step proves the evaluation claim $Q(z) = v$ using stacked **BaseFold**, the multilinear variant of FRI.

Protocol. BaseFold combines the standard FRI proximity test with a sumcheck that establishes the multilinear evaluation of the committed oracle. Recall that Q is described as a table of height `dense_length` and width `dense_batch`. Let $n := \lceil \log_2(\texttt{dense_batch}) \rceil$, $m := \log_2(\texttt{dense_length})$. Decompose $z = (z_{stack}, z_{col})$, where $z_{stack} \in \mathbb{E}^m, \quad z_{col} \in \mathbb{E}^n$.

1. **Columnwise claims.** The prover sends multilinear evaluation claims of the columns of Q at z_{stack}.

2. **Batching step.** The verifier samples randomness $\tau \in \mathbb{E}^n$. The prover takes the random linear combination of the columns of Q via the coefficients $\hat{\mathrm{eq}}_\tau(\cdot)$. The prover and verifier take the same linear combination of the column evaluation claims. In the remaining steps, prover and verifier engage in the BaseFold protocol for the random linear combination.

3. **Folding rounds.** In each round the prover sends a sumcheck message and folds the current polynomial by a factor of 2 (as specified by `fri_folding_factors`), using the next sumcheck challenge as the folding randomness. The folded oracle is committed via Reed-Solomon encoding and a fresh Merkle tree.

4. **Early stop.** Folding stops when length of the R-S encoded oracle reaches `fri_early_stop_degree`. The prover sends the remaining coefficients in the clear.

[6] `https://eprint.iacr.org/2025/917`

5. **Query phase.** The verifier samples `num_queries` query positions. For each query the prover opens the corresponding leaf in every folding-round Merkle tree. Proof-of-work grinding of `grinding_query_phase` bits precedes this phase.